Akademia Górniczo-Hutnicza im. Stanisława Staszica w Krakowie
| Wydział Inżynierii Mechanicznej i Robotyki | Katedra Robotyki i Mechatroniki

Mechanika i Budowa Maszyn | I stopień | semestr VII | 2026/2027

Podstawy Mechatroniki

Instrukcja 3:

Symulacja układów wieloczłonowych w programie SimWise – implementacja równań ruchu w algorytmie sterowania Matlab/Simulink

Nauczysz się:
Uwzględniać zależności dynamiczne w pracy symulowanego układu wieloczłonowego.

Dodatkowe materiały:

- materiały do zajęć: [*link*].

Kierunkowe efekty kształcenia pokryte w tej instrukcji:
U2, U3

Koordynator przedmiotu:
dr hab. inż. Michał Mańka, prof. AGH, mmanka@agh.edu.pl

Autor instrukcji:
dr inż. Jakub Bryła, jakub.bryla@agh.edu.pl

1. Informacje wstępne

Dla analizowanego projektu manipulatora SCARA przeprowadziliśmy symulacje odpowiedzialne za realizację zadania kinematyki prostej i odwrotnej. Tym samym potwierdziliśmy zakres ruchów / obszar pracy robota oraz zależność pozycji efektora od zmiennych przegubowych.

Kolejnym etapem projektu będzie uwzględnienie parametrów mających wpływ na właściwości dynamiczne układu mechanicznego. Poprzez uwzględnienie masy manipulatora i komponentów środowiska oraz określenie prędkości przegubowych w analizie zostanie uwzględnione zjawisko bezwładności.

2. Wprowadzenie

2.1. Równania ruchu

Równania ruchu pozwalają określić sposób przejścia z pozycji **A** do **B**. Zazwyczaj przyjmują one zależność od czasu:

$$q(t) = q_A + (q_B - q_A)\, s(t), \qquad t \in [0, T]$$

gdzie:

- q_A – konfiguracja początkowa,
- q_B – konfiguracja końcowa,
- T – czas przejazdu,
- $s(t)$ – funkcja profilująca ruch, spełniająca założenia:

$$s(0) = 0, \quad s(T) = 1$$

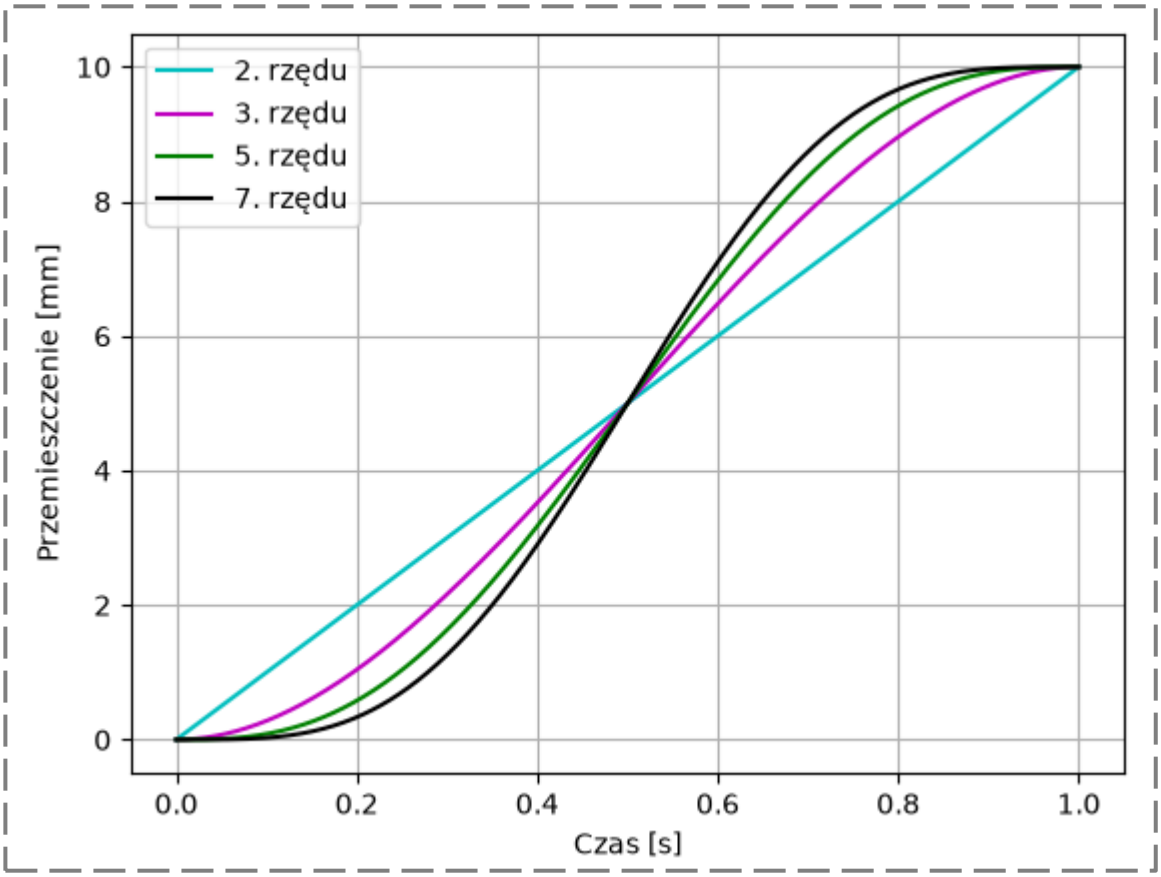
Poniżej przedstawiono równania ruchu dla funkcji profilujących w postaci wielomianów różnego rzędu. Wprowadzono zmienne pomocnicze:

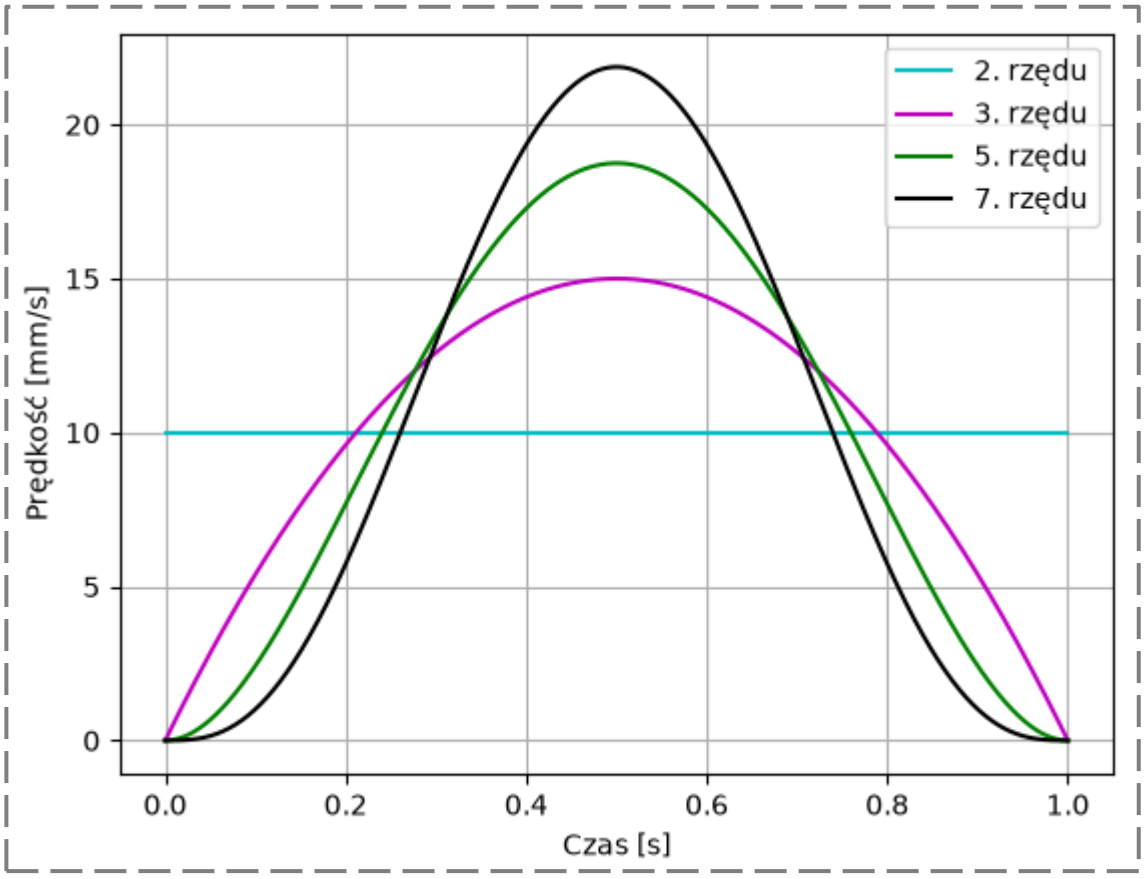
$$\tau = \frac{t}{T}$$

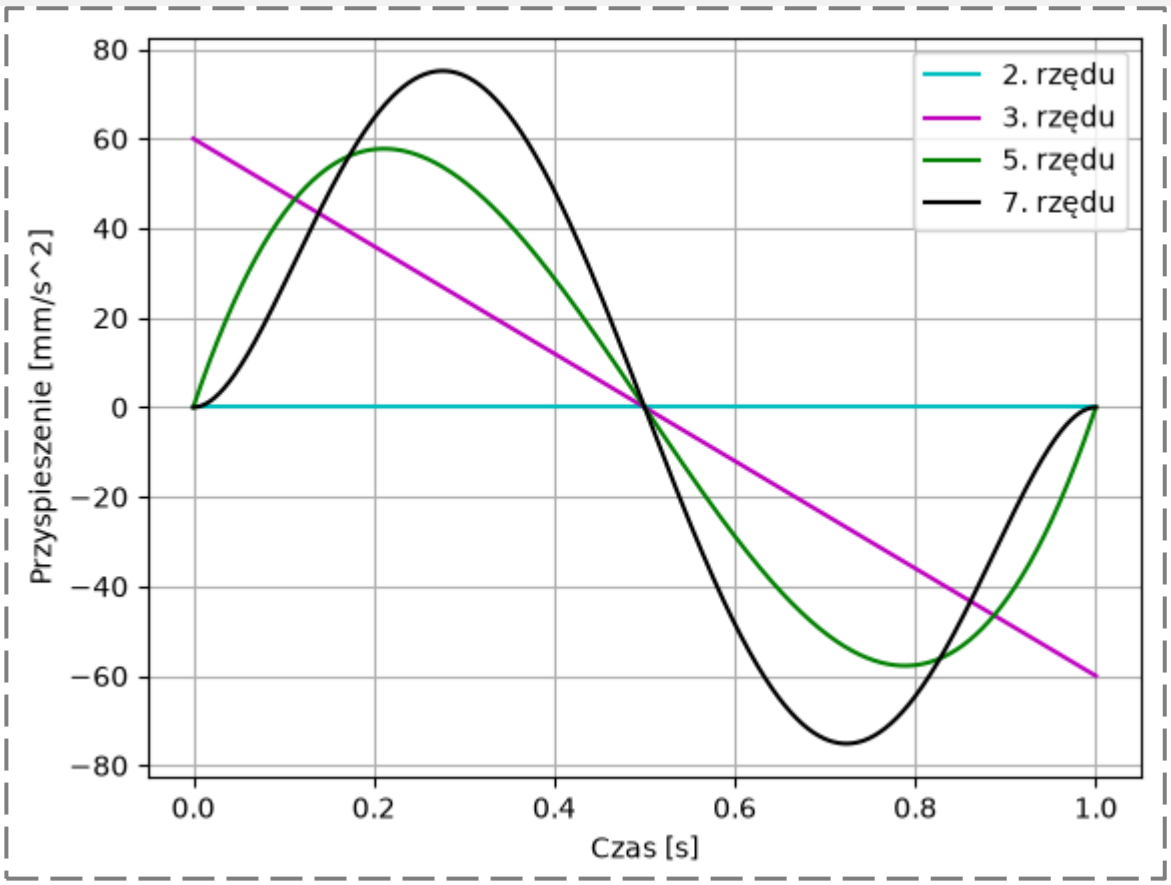
$$\Delta q = q_B - q_A$$

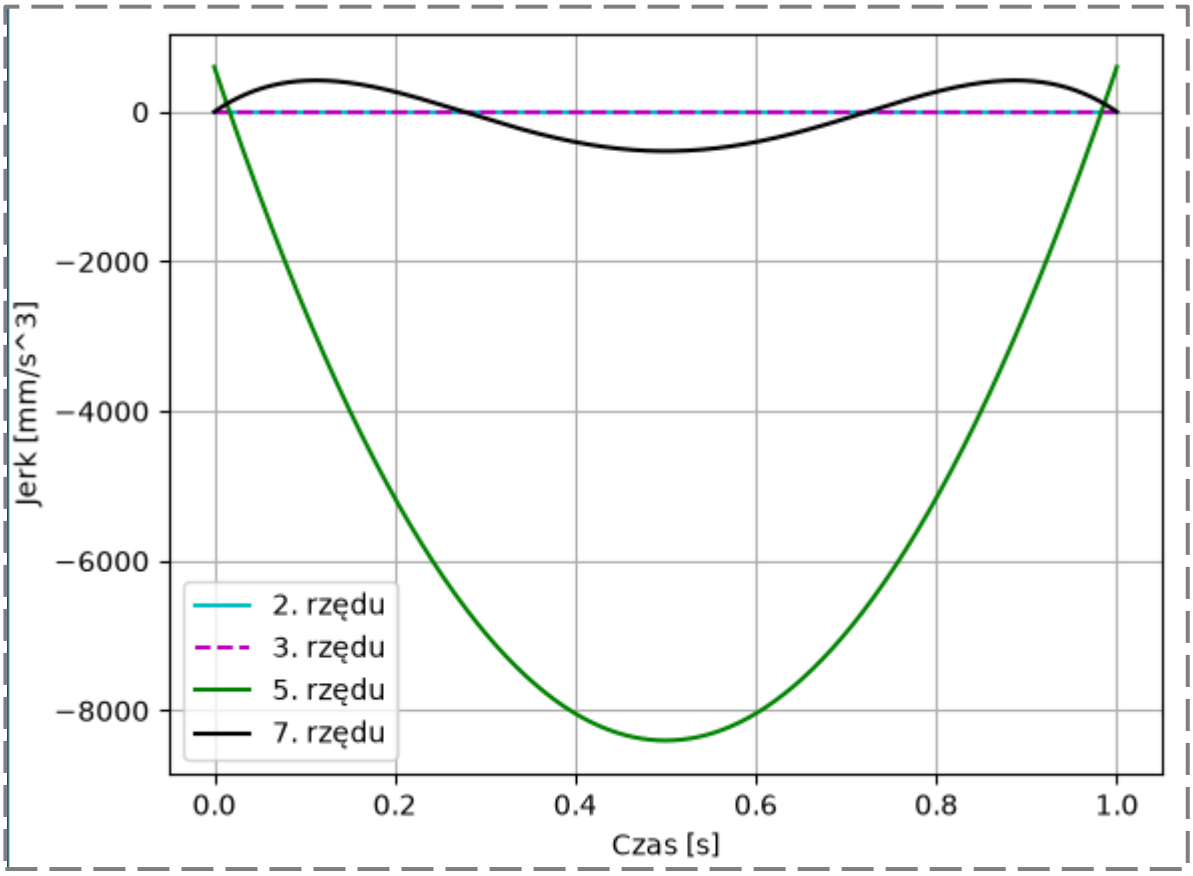
	Ruch liniowy – w. II rzędu	W. III rzędu	Min-jerk – w. V rzędu	Min-snap – w. VII rzędu
F. profilująca ruch $s(t)$	τ	$3\tau^2 - 2\tau^3$	$10\tau^3 - 15\tau^4 + 6\tau^5$	$35\tau^4 - 84\tau^5 + 70\tau^6 - 20\tau^7$
F. prędkości $\dot{q}(t)$	$\frac{\Delta q}{T}$	$\Delta q \left(\frac{6t}{T^2} - \frac{6t^2}{T^3}\right)$	$\Delta q\ 30\tau^2(1 - 2\tau + \tau^2)$	$\frac{140\,\Delta q}{T}\tau^3(1-\tau)^3$
F. przyśpieszenia $\ddot{q}(t)$	0	$\Delta q \left(\frac{6}{T^2} - \frac{12t}{T^3}\right)$	$\Delta q\ 60\tau\,(1 - 3\tau + 2\tau^2)$	$\frac{\Delta q}{T^2}\ 420\tau^2(1 - 4\tau + 5\tau^2 - 2\tau^3)$
Jerk $\dddot{q}(t)$	0	0	$\frac{\Delta q}{T}\ 60\,(1 - 60\tau + 60\tau^2)$	$\frac{420\,\Delta q}{T^3}\ 2\tau\,(1 - 6\tau + 10\tau^2 - 5\tau^3)$
Czas przejazdu	$T \geq \frac{\Delta q}{V_{max}}$	$T \geq \frac{3\Delta q}{2V_{max}}$ $T \geq \sqrt{6\frac{\Delta q}{a_{max}}}$	$T \geq \frac{15\Delta q}{8V_{max}}$ $T \geq \sqrt{\frac{10}{\sqrt{3}}\frac{\Delta q}{a_{max}}}$ $T \geq \sqrt[3]{60\frac{\Delta q}{j_{max}}}$	$T \geq \frac{35\Delta q}{16V_{max}}$ $T \geq \sqrt{\frac{105}{16\sqrt{5}}\frac{\Delta q}{a_{max}}}$ $T \geq \sqrt[3]{\frac{30(3+\sqrt{5})}{8}\frac{\Delta q}{j_{max}}}$

Dla zobrazowania charakterystyk w zależności od funkcji profilującej ruch poniżej przedstawiono wyniki dla T=1, $q_A = 0$ i $q_B = 10$.


2. rzędu
3. rzędu
5. rzędu
7. rzędu
Przemieszczenie [mm]
Czas [s]
0
2
4
6
8
10
0.0
0.2
0.4
0.6
0.8
1.0


2. rzędu
3. rzędu
5. rzędu
7. rzędu
Prędkość [mm/s]
Czas [s]
0
5
10
15
20
0.0
0.2
0.4
0.6
0.8
1.0


80
60
40
20
0
−20
−40
−60
−80
Przyspieszenie [mm/s^2]
0.0
0.2
0.4
0.6
0.8
1.0
Czas [s]
2. rzędu
3. rzędu
5. rzędu
7. rzędu


0
−2000
−4000
−6000
−8000
Jerk [mm/s^3]
0.0
0.2
0.4
0.6
0.8
1.0
Czas [s]
2. rzędu
3. rzędu
5. rzędu
7. rzędu

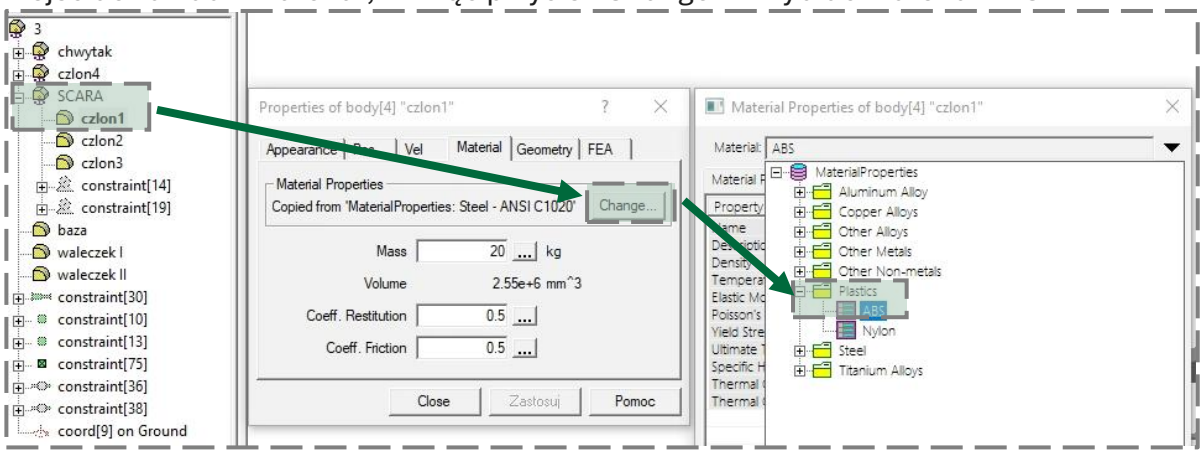
2.2. Założenia

Mając na uwadze maksymalną prędkość obrotową silników krokowych oraz brak przekładni w zamodelowanym manipulatorze, ograniczenia prędkości zostaną narzucone na ruch efektora. Oznacz to, że czas potrzebny do zmiany konfiguracji robota będzie wyznaczany we współrzędnych XYZ. Czas przejazdu będzie wyznaczany z kryterium prędkości i przyśpieszenia dla składowych XYZ ruchu efektora i otwarcia szczęk.
Następnie odpowiednie funkcje profilujące ruch zostaną zaimplementowane dla parametrów przegubowych, gwarantując zamierzony charakter pracy silników.

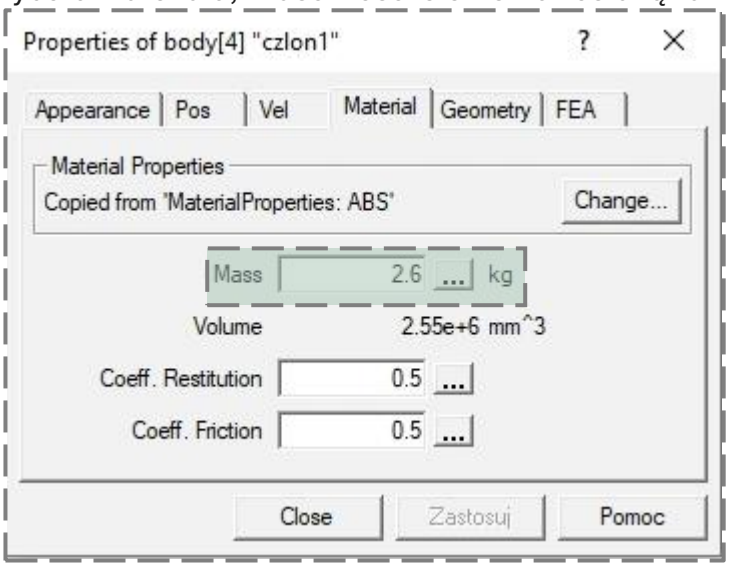
3. Zadania

ZADANIE 3.1 – definicja materiału dal poszczególnych części manipulatora

W pierwszej kolejności w programie SimWise należy uwzględnić rodzaj materiału z jakiego wykonano poszczególne części manipulatora. W tym celu należy dwukrotnie kliknąć lewym przyciskiem myszy na element *czlon1* w podfolderze *Scara*. W otwartym oknie należy przejść do zakładki *Material*, kliknąć przycisk *Change...* i wybrać materiał *ABS*:



Po zatwierdzeniu wyboru materiału, właściwości elementu zostaną zaktualizowane:



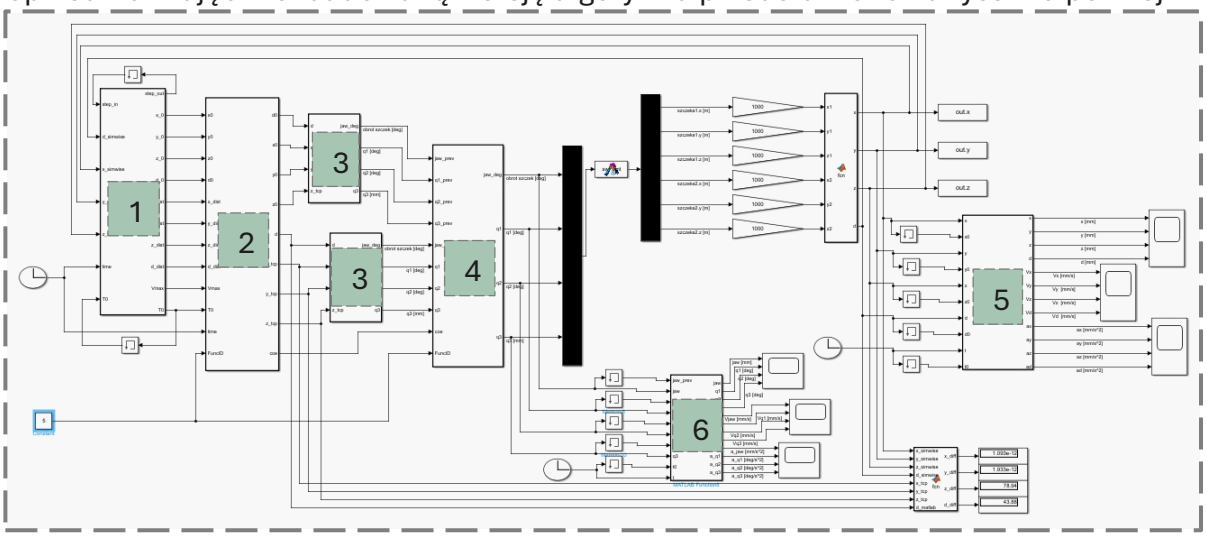
Powyższe operacje należy powtórzyć dla następujących części:

- SCARA/czlon2,
- SCARA/czlon3,

- czlon4/mocowanie chwytaka,
- czlon4/zaslepka,
- chwytak/link,
- chwytak/link Nema,
- chwytak/link2 I,
- chwytak/link2 II,
- chwytak/obudowa,
- chwytak/szczeka I,
- chwytak/szczeka II.

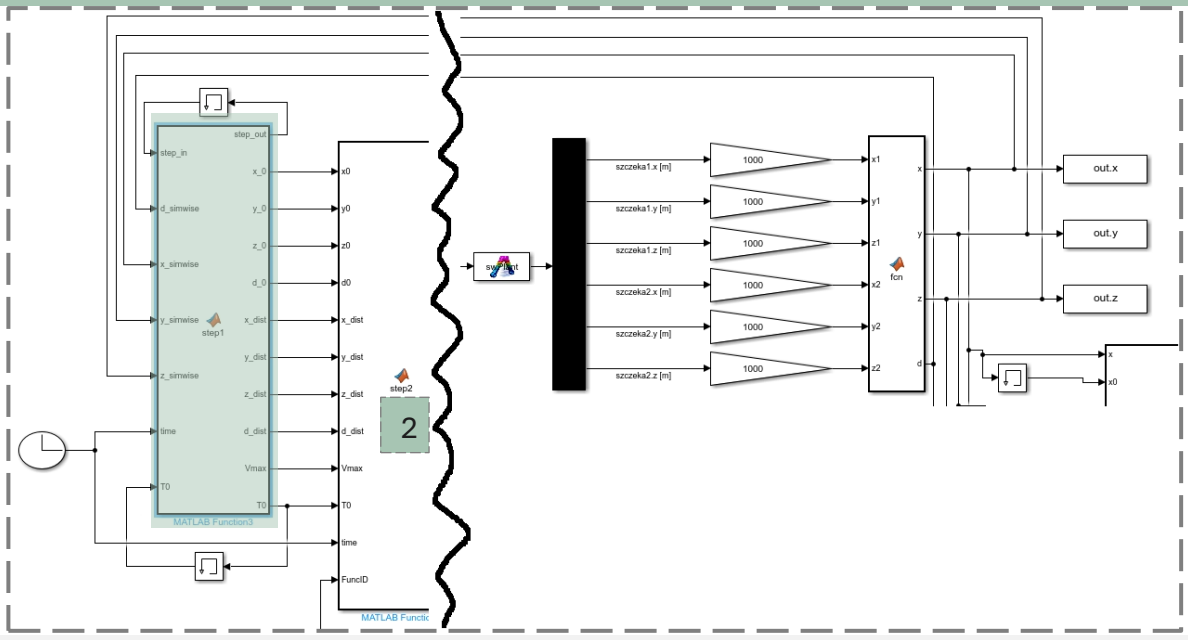
ZADANIE 3.2 – implementacja równań ruchu

Zadanie wymaga aktualizacji projektu sterowania w programie Matlab/Simulink z poprzednich zajęć. Rozbudowaną wersję algorytmu przedstawiono na rysunku poniżej.



Powyższy schemat blokowy obrazuje niedogodność programowania bloczkowego – w dalszym ciągu jest to stosunkowo nierozbudowany program, w którym jednak aktualna liczba połączeń między blokami utrudnia zapanowanie nad ładem w projekcie. Przyjrzyjmy się poszczególnym zmianom w bloczkach programistycznych.

Blok 1



Blok odpowiada za zdefiniowanie kolejnych konfiguracji manipulatora oraz weryfikację czy obecnie żądana pozycja została osiągnięta. W celu skrócenia czasu symulacji wprowadzono różne prędkości ruchu członów manipulatora w układzie kartezjańskim. Powyższe działanie wynika z faktu, że ruch jałowy robota może odbywać się znacznie szybciej – udane symulacje przeprowadzono z prędkością 300mm/s. W kluczowym momencie przenoszenia pionu przez chwytak zwalniano do 50mm/s. Należy się upewnić, że bloczek „*Memory*" powiązany ze zmienną *T0* przyjmuje wartość domyślną 0.

Kod bloczka 1:

```
function [step_out, x_0, y_0, z_0, d_0, x_dist, y_dist, z_dist, d_dist, Vmax, T0] = step1(step_in, d_simwise, x_simwise, y_simwise, z_simwise, time, T0)

x_list = [0, -350, -350];
y_list = [-350, 0, 0];
z_list = [80, 80, 60];
d_list = [45, 45, 0];
V_list = [300, 100, 50];

%% TCP
x_tcp = x_list(step_in);
y_tcp = y_list(step_in);
z_tcp = z_list(step_in);
d = d_list(step_in);
Vmax = V_list(step_in);

%% error
error = 1;
x_error = abs(x_tcp - x_simwise);
y_error = abs(y_tcp - y_simwise);
z_error = abs(z_tcp - z_simwise);
d_error = abs(d - d_simwise);

%% change conf
step_out = step_in;

if (x_error <= error) && (y_error <= error) && (z_error <= error) && (d_error <= error)
    if length(x_list) > step_in
        step_out = step_out + 1;
        T0 = time;

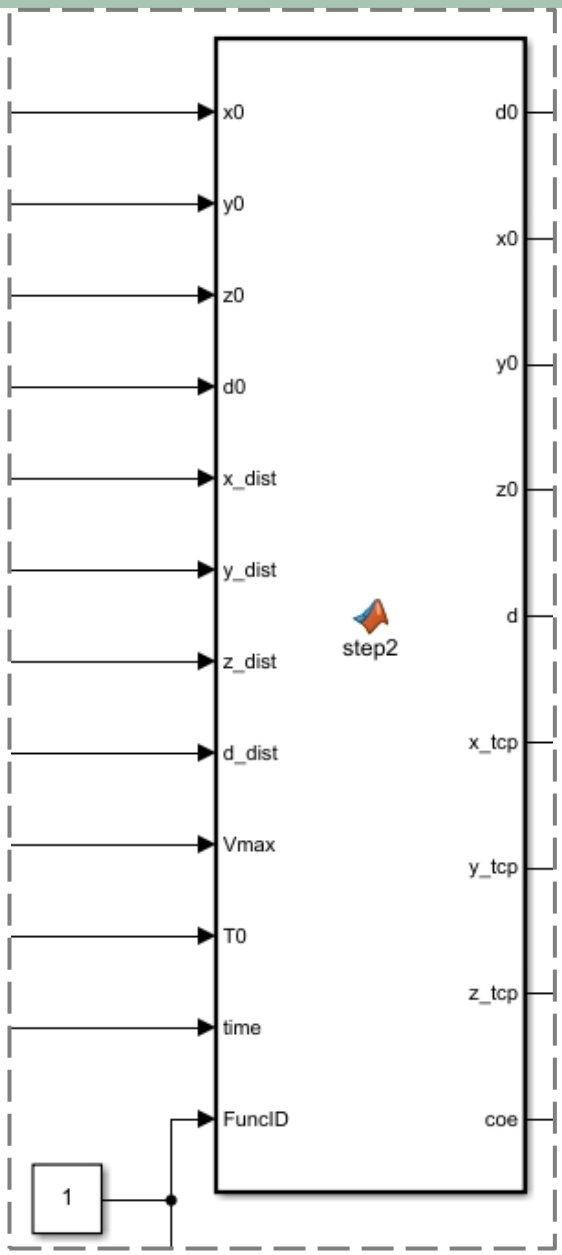
        x_tcp = x_list(step_out);
        y_tcp = y_list(step_out);
        z_tcp = z_list(step_out);
        d = d_list(step_out);
        Vmax = V_list(step_out);
    end
end

%% distance
if step_out == 1
    x_0 = 0;
    y_0 = -350;
    z_0 = 0;
    d_0 = 0;
else
    x_0 = x_list(step_out - 1);
    y_0 = y_list(step_out - 1);
    z_0 = z_list(step_out - 1);
```

```
49      d_0 = d_list(step_out - 1);
50  end
51
52  x_dist = x_tcp - x_0;
53  y_dist = y_tcp - y_0;
54  z_dist = z_tcp - z_0;
55  d_dist = d - d_0;
```

W liniach 3-7 należy podać konfiguracje manipulatora gwarantujące umieszczenie jednego pionu na drugim – seria kolejnych pozycji wyznaczana była na poprzednich zajęciach.

Blok 2



Blok odpowiada za policzenie czasu przejazdu od pozycji początkowej [d0, x0, y0, z0] do pozycji [d, x_tcp, y_tcp, z_tcp] w zależności od wyboru funkcji profilującej ruch. Dla wygody pracy zaimplementowano funkcję warunkową – w zależności od wartości parametru *FuncID* wybierana jest odpowiednia postać wielomianu:

- FuncID = 1 → wielomian II rzędu,
- FuncID = 3 → wielomian III rzędu,
- FuncID = 5 → wielomian V rzędu,
- FuncID = 7 → wielomian VII rzędu.

Na podstawie wyznaczonego czasu przejazdu oraz czasu trwania ruchu *(time – T0)* określa się współczynnik pokonanej drogi *coe*.

Kod bloku 2:

```
function [d0, x0, y0, z0, d, x_tcp, y_tcp, z_tcp, coe] = step2(x0, y0, z0, d0, x_dist, y_dist, z_dist, d_dist, Vmax, T0, time, FuncID)

%% time to reach next step
a_max = 1500;

    % Vconst
if (FuncID == 1) || (FuncID == 10)
    tx = abs(x_dist) / Vmax;
    ty = abs(y_dist) / Vmax;
    tz = abs(z_dist) / Vmax;
    td = abs(d_dist) / Vmax;

    t_span = max([tx, ty, tz, td]);

    % 3. stopnia
elseif (FuncID == 3) || (FuncID == 30)
    tx_1 = (3 * abs(x_dist) / (2 * Vmax));
    ty_1 = (3 * abs(y_dist) / (2 * Vmax));
    tz_1 = (3 * abs(z_dist) / (2 * Vmax));
    td_1 = (3 * abs(d_dist) / (2 * Vmax));

    tx_2 = sqrt(6 * abs(x_dist) / a_max);
    ty_2 = sqrt(6 * abs(y_dist) / a_max);
    tz_2 = sqrt(6 * abs(z_dist) / a_max);
    td_2 = sqrt(6 * abs(d_dist) / a_max);

    t_span = max([tx_1, ty_1, tz_1, td_1, tx_2, ty_2, tz_2, td_2]);

    % 5. stopnia - min-jerk
elseif (FuncID == 5) || (FuncID == 50)
    tx_1 = (15 * abs(x_dist) / (8 * Vmax));
    ty_1 = (15 * abs(y_dist) / (8 * Vmax));
    tz_1 = (15 * abs(z_dist) / (8 * Vmax));
    td_1 = (15 * abs(d_dist) / (8 * Vmax));

    tx_2 = sqrt(10 * abs(x_dist) / (sqrt(3) * a_max));
    ty_2 = sqrt(10 * abs(y_dist) / (sqrt(3) * a_max));
    tz_2 = sqrt(10 * abs(z_dist) / (sqrt(3) * a_max));
    td_2 = sqrt(10 * abs(d_dist) / (sqrt(3) * a_max));

    t_span = max([tx_1, ty_1, tz_1, td_1, tx_2, ty_2, tz_2, td_2]);

    % 7. stopnia - min-span
elseif (FuncID == 7) || (FuncID == 70)
    tx_1 = (35 * abs(x_dist) / (16 * Vmax));
    ty_1 = (35 * abs(y_dist) / (16 * Vmax));
    tz_1 = (35 * abs(z_dist) / (16 * Vmax));
    td_1 = (35 * abs(d_dist) / (16 * Vmax));

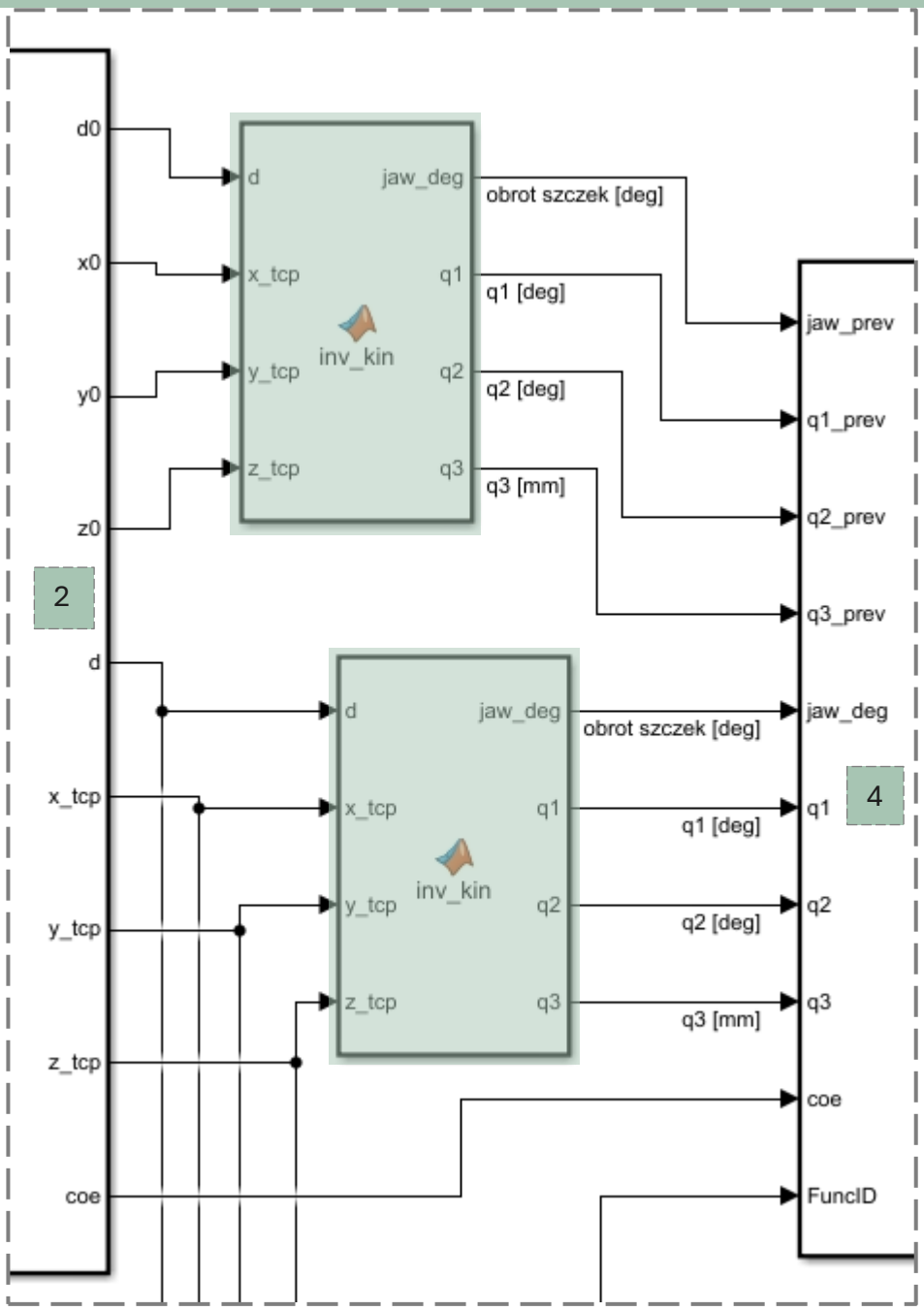
    tx_2 = sqrt(105 * abs(x_dist) / (16 * sqrt(5) * a_max));
    ty_2 = sqrt(105 * abs(y_dist) / (16 * sqrt(5) * a_max));
    tz_2 = sqrt(105 * abs(z_dist) / (16 * sqrt(5) * a_max));
    td_2 = sqrt(105 * abs(d_dist) / (16 * sqrt(5) * a_max));

    t_span = max([tx_1, ty_1, tz_1, td_1, tx_2, ty_2, tz_2, td_2]);

```

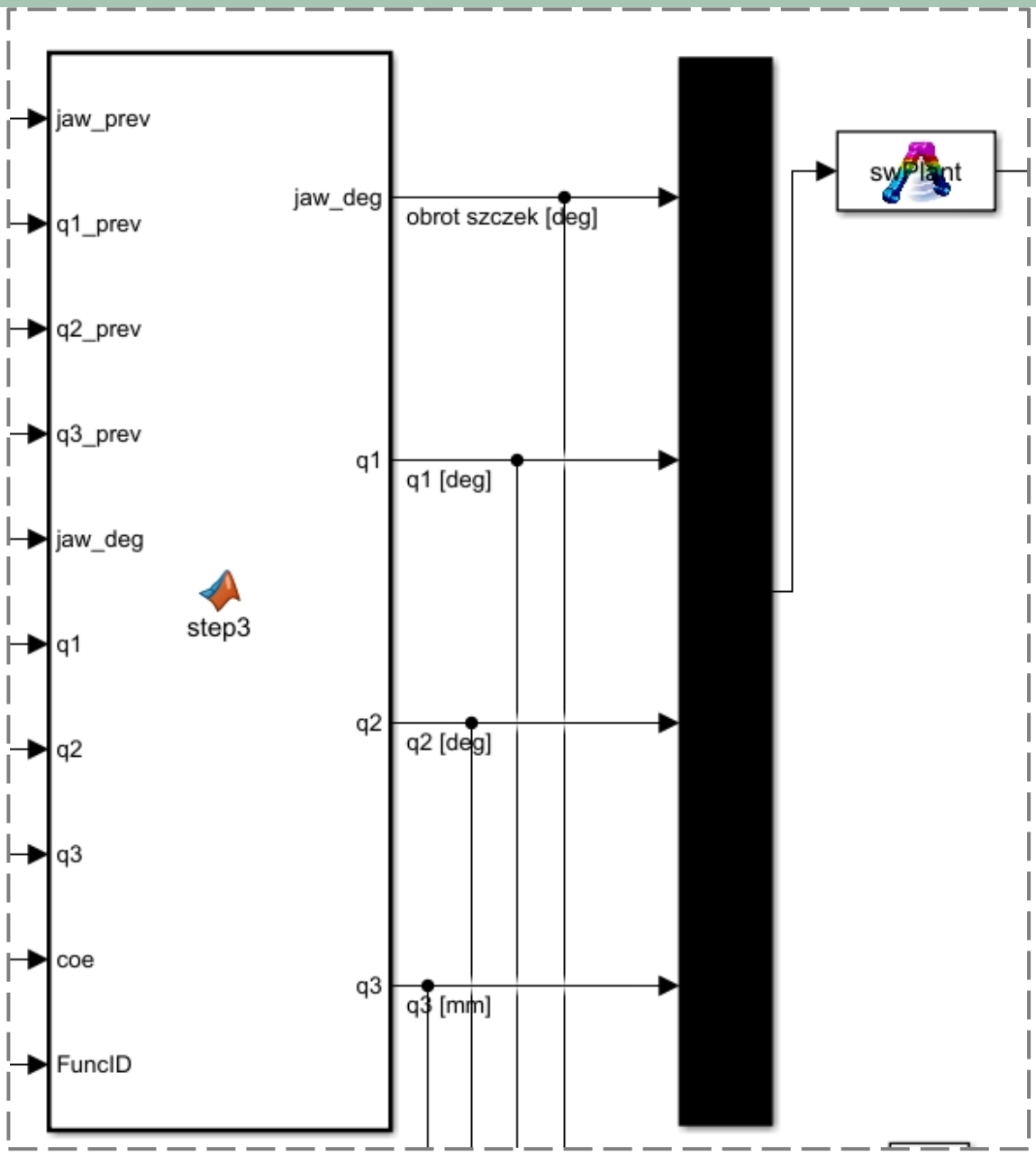
```
57  else
58      t_span = 10000;
59  end
60
61  %% get TCP/coe at t
62  dt = time - T0;
63
64  switch FuncID
65      % Vconst
66      case 1
67          coe = dt / t_span;
68
69      case 10
70          coe = dt / t_span;
71
72      % 3. stopnia
73      case 3
74          coe = 3 * (dt / t_span)^2 - 2 * (dt / t_span)^3;
75
76      case 30
77          coe = 3 * (dt / t_span)^2 - 2 * (dt / t_span)^3;
78
79      % 5. stopnia
80      case 5
81          coe = 10 * (dt / t_span)^3 - 15 * (dt / t_span)^4 + 6 * (dt /
    t_span)^5;
82
83      case 50
84          coe = 10 * (dt / t_span)^3 - 15 * (dt / t_span)^4 + 6 * (dt /
    t_span)^5;
85
86      % 7. stopnia
87      case 7
88          coe = 35 * (dt / t_span)^4 - 84 * (dt / t_span)^5 + 70 * (dt /
    t_span)^6 - 20 * (dt / t_span)^7;
89
90      case 70
91          coe = 35 * (dt / t_span)^4 - 84 * (dt / t_span)^5 + 70 * (dt /
    t_span)^6 - 20 * (dt / t_span)^7;
92
93      otherwise
94          coe = 1;
95  end
96
97  %% results
98  if coe > 1
99      coe = 1;
100 end
101
102 if FuncID < 10
103     x_tcp = x0 + x_dist;
104     y_tcp = y0 + y_dist;
105     z_tcp = z0 + z_dist;
106     d = d0 + d_dist;
107 else
108     x_tcp = x0 + x_dist * coe;
109     y_tcp = y0 + y_dist * coe;
110     z_tcp = z0 + z_dist * coe;
111     d = d0 + d_dist * coe;
112 end
```

Blok 3



Kod bloku odpowiada za policzenie zadania kinematyki odwrotnej. Nie został on zmieniony względem poprzedniej instrukcji. Blok został natomiast skopiowany w celu bieżącego obliczania parametrów przegubowych dla poprzedniej konfiguracji.

Blok 4



W bloku na podstawie poprzedniej i obecnej konfiguracji manipulatora, współczynnika *coe* i wybranej funkcji profilującej ruch (*FuncID*) określa się żądaną wartość parametrów przegubowych w zadanej chwili czasowej.

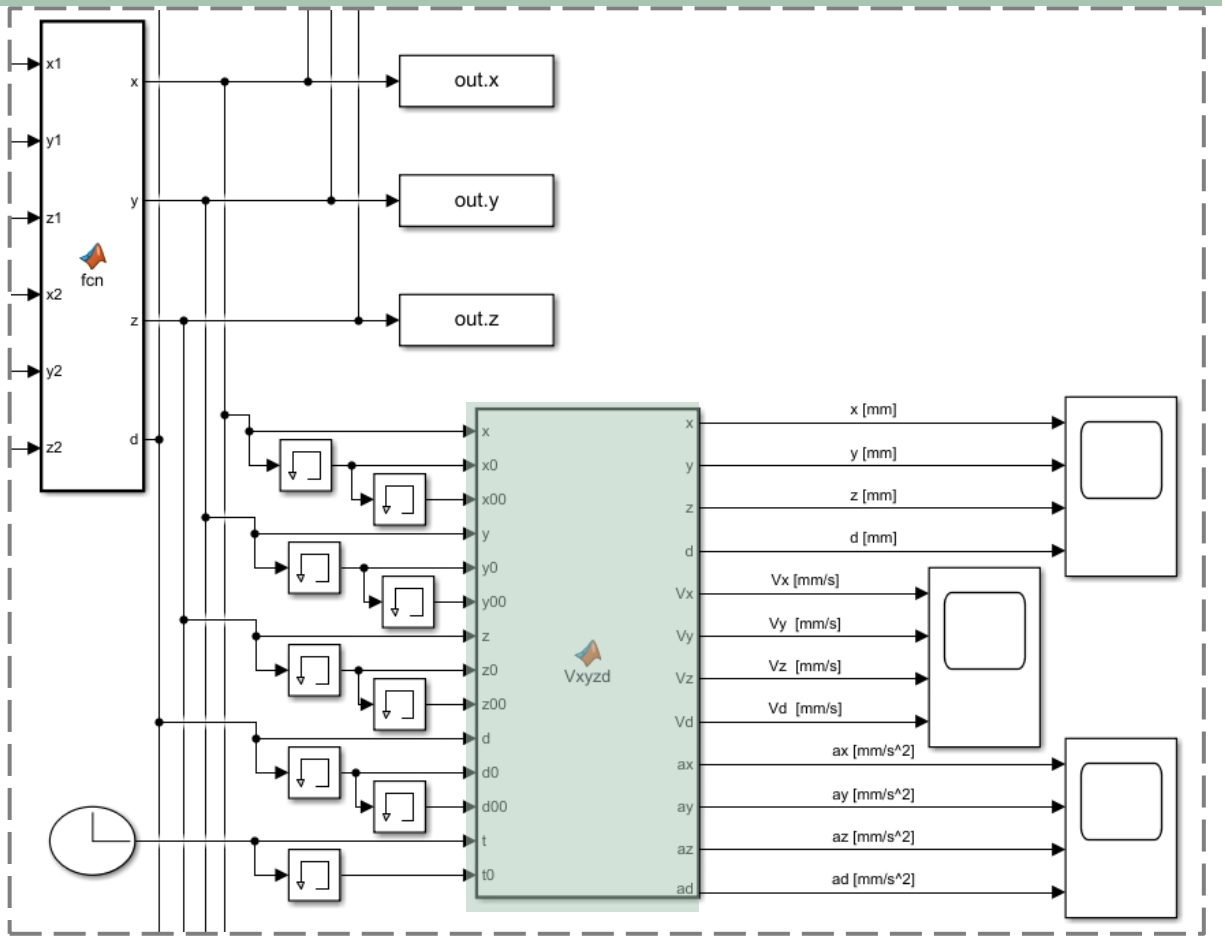
Kod bloku 4:

```
function [jaw_deg, q1, q2, q3] = step3(jaw_prev, q1_prev, q2_prev, q3_prev,
jaw_deg, q1, q2, q3, coe, FuncID)

%%

if FuncID < 10
    q1 = q1_prev + (q1 - q1_prev) * coe;
    q2 = q2_prev + (q2 - q2_prev) * coe;
    q3 = q3_prev + (q3 - q3_prev) * coe;
    jaw_deg = jaw_prev + (jaw_deg - jaw_prev) * coe;
end
```

Blok 5



Kod bloku odpowiada za przekazanie pozycji oraz obliczenie prędkości i przyśpieszenia punktu TCP (skrajnego punktu chwytaka) w układzie współrzędnych kartezjańskich. Dane przekazywane są do odpowiednich wykresów.

Kod bloku 5:

```
function [x, y, z, d, Vx, Vy, Vz, Vd, ax, ay, az, ad] =Vxyzd(x, x0, x00, y, y0, y00, z, z0, z00, d, d0, d00, t, t0)

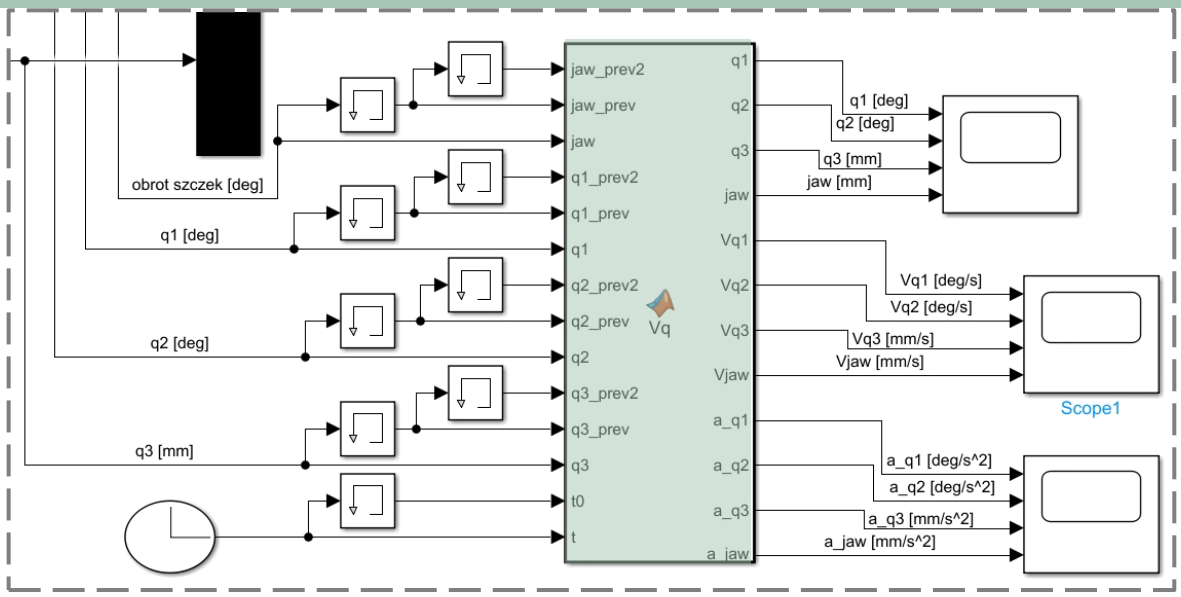
t_span = t - t0;

Vx = (x - x0) / t_span;
Vy = (y - y0) / t_span;
Vz = (z - z0) / t_span;
Vd = (d - d0) / t_span;

Vx_prev = (x0 - x00) / t_span;
Vy_prev = (y0 - y00) / t_span;
Vz_prev = (z0 - z00) / t_span;
Vd_prev = (d0 - d00) / t_span;

ax = (Vx - Vx_prev) / t_span;
ay = (Vy - Vy_prev) / t_span;
az = (Vz - Vz_prev) / t_span;
ad = (Vd - Vd_prev) / t_span;
```

Blok 6



Podobnie jak dla bloku 5, kod bloku 6 odpowiada za przekazanie pozycji oraz wyznaczenie prędkości i przyśpieszenia – tym razem dla współrzędnych przegubowych.

Kod bloku 6:

```
function [q1, q2, q3, jaw, Vq1, Vq2, Vq3, Vjaw, a_q1, a_q2, a_q3, a_jaw] = Vq(jaw_prev2, jaw_prev, jaw, q1_prev2, q1_prev, q1, q2_prev2, q2_prev, q2, q3_prev2, q3_prev, q3, t0, t)

dt = t - t0;
Vjaw = (jaw - jaw_prev) / dt;
Vq1 = (q1 - q1_prev) / dt;
Vq2 = (q2 - q2_prev) / dt;
Vq3 = (q3 - q3_prev) / dt;

Vjaw_prev = (jaw_prev - jaw_prev2) / dt;
Vq1_prev = (q1_prev - q1_prev2) / dt;
Vq2_prev = (q2_prev - q2_prev2) / dt;
Vq3_prev = (q3_prev - q3_prev2) / dt;

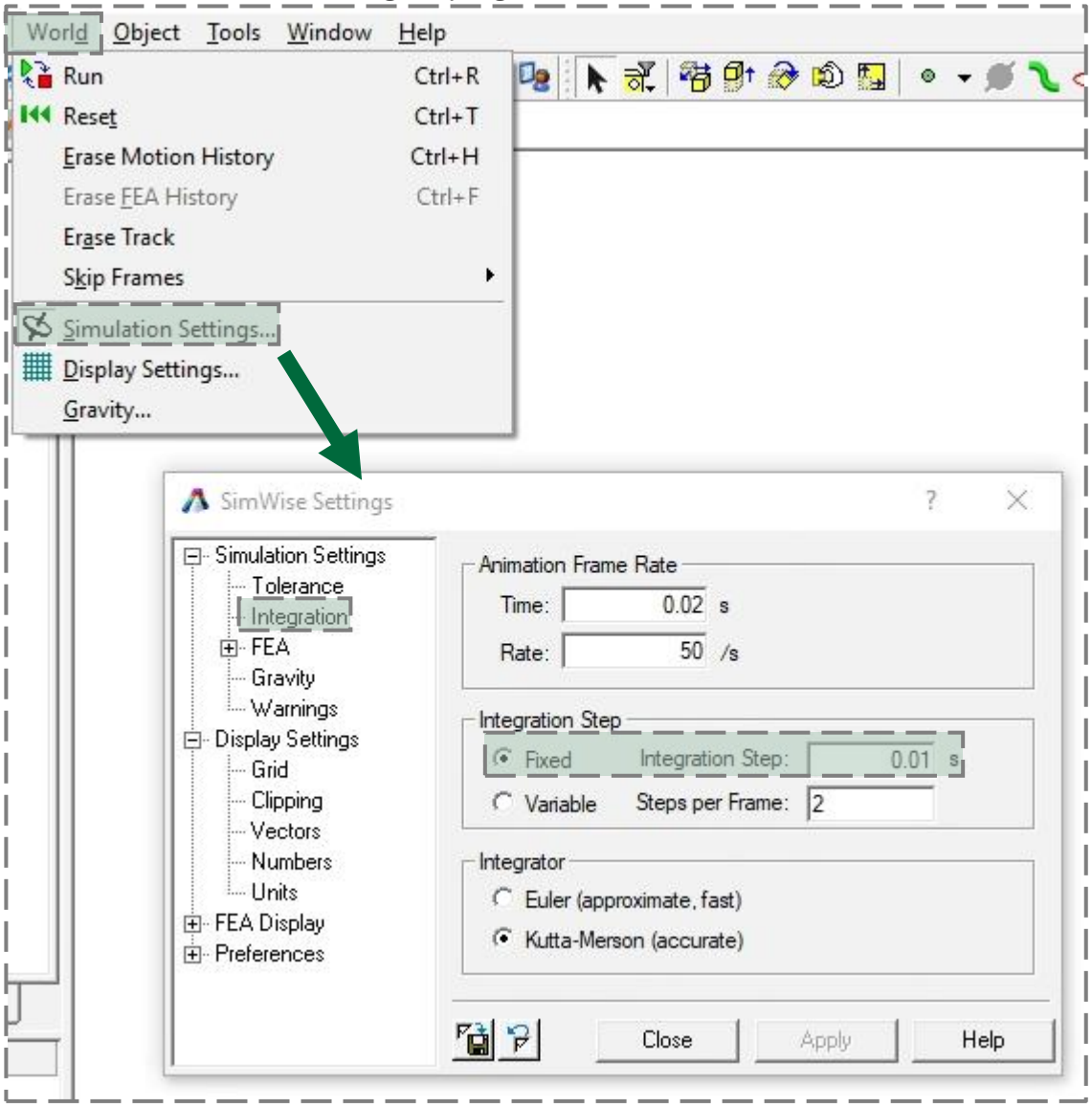
a_jaw = (Vjaw - Vjaw_prev) / dt;
a_q1 = (Vq1 - Vq1_prev) / dt;
a_q2 = (Vq2 - Vq2_prev) / dt;
a_q3 = (Vq3 - Vq3_prev) / dt;
```

ZADANIE 3.3 – studium przypadku

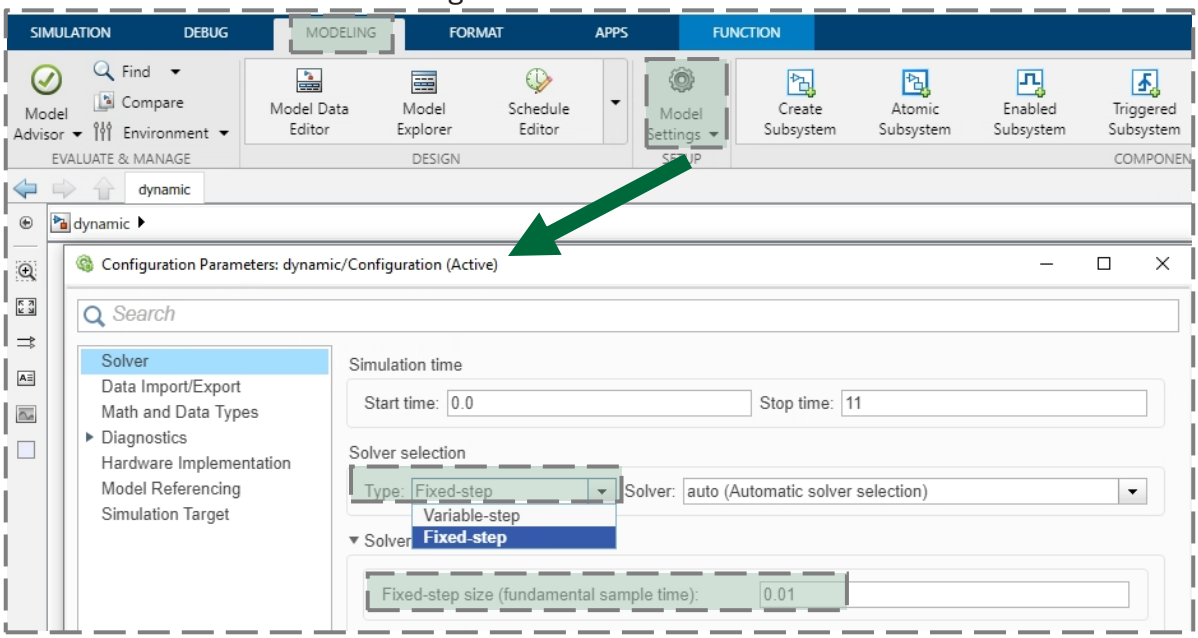
Po wprowadzonych zmianach w projekcie możemy wrócić do próby ustawienia jednego pionu na drugim. Zanim jednak przejdziemy do uruchomienia symulacji zmienimy ustawienia dotyczące kroku czasowego. Ustawimy stały krok czasowy równy 0,01s – dla stałego kroku czasowego obliczenia trwają nieco krócej, a sama symulacja charakteryzuje się większą stabilnością.

Kwestie związane ze zbieżnością wyników i walidacją modelu są kluczowe dla symulacji numerycznych i wiarygodności uzyskanych wyników. „Dostrajanie" modelu stanowi jeden z najbardziej pracochłonnych etapów całego procesu symulacyjnego, często będąc kwestią charakterystyczną dla konkretnego środowiska symulacyjnego – poprawne ustawienie parametrów symulacji wymaga doskonałej znajomości programu i często wiedzy teoretycznej na temat zaimplementowanych algorytmów numerycznych.

Zmiana ustawień kroku czasowego w programie SimWise:



Zmiana ustawień kroku czasowego w Matlab/Simulink:



Uruchom symulację, dla określonej na poprzednich zajęciach serii konfiguracji manipulatora. Postaraj się uzyskać pozytywny rezultat dla przynajmniej **dwóch** różnych funkcji profilujących ruch – postać funkcji zmienisz ustawiając wartość zmiennej *FuncID*:

- FuncID = 1 → wielomian II rzędu,
- FuncID = 3 → wielomian III rzędu,
- FuncID = 5 → wielomian V rzędu,
- FuncID = 7 → wielomian VII rzędu.

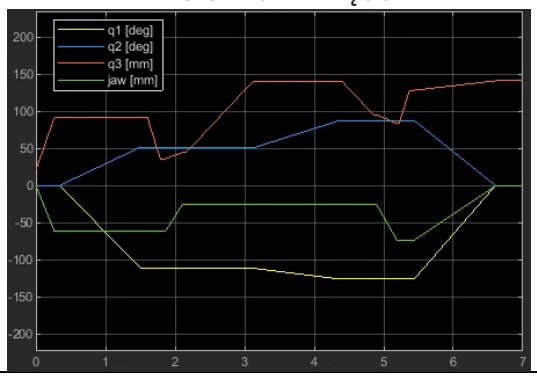
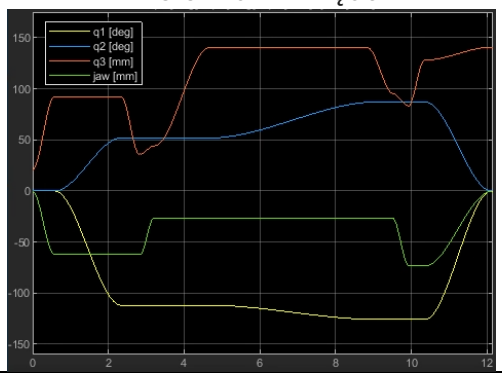
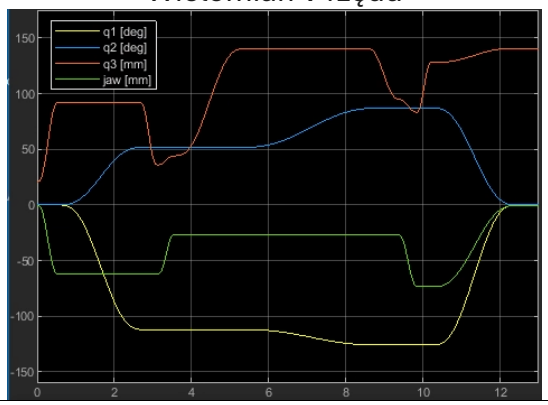
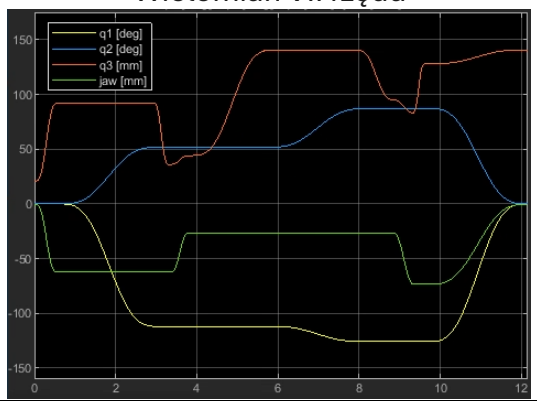
4. Podsumowanie

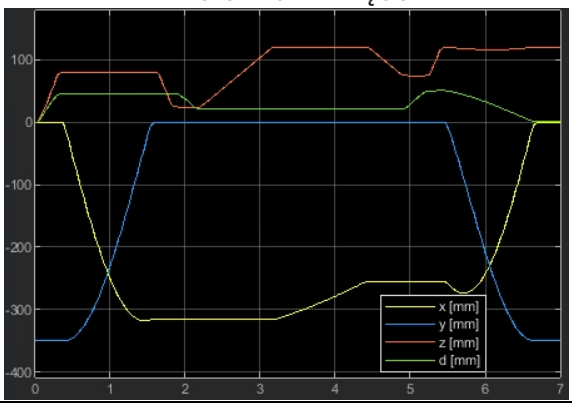
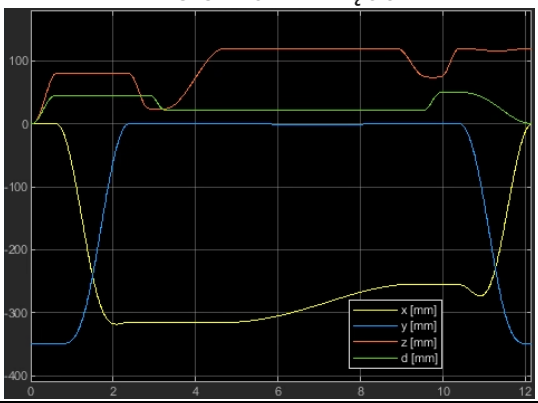
Poniżej przedstawiono przykładowe wyniki uzyskane przez prowadzącego. Kluczowe jest porównanie różnic wynikających z zastosowanych funkcji ruchu.
Wykonane zadania w niniejszej instrukcji pozwalają zweryfikować:

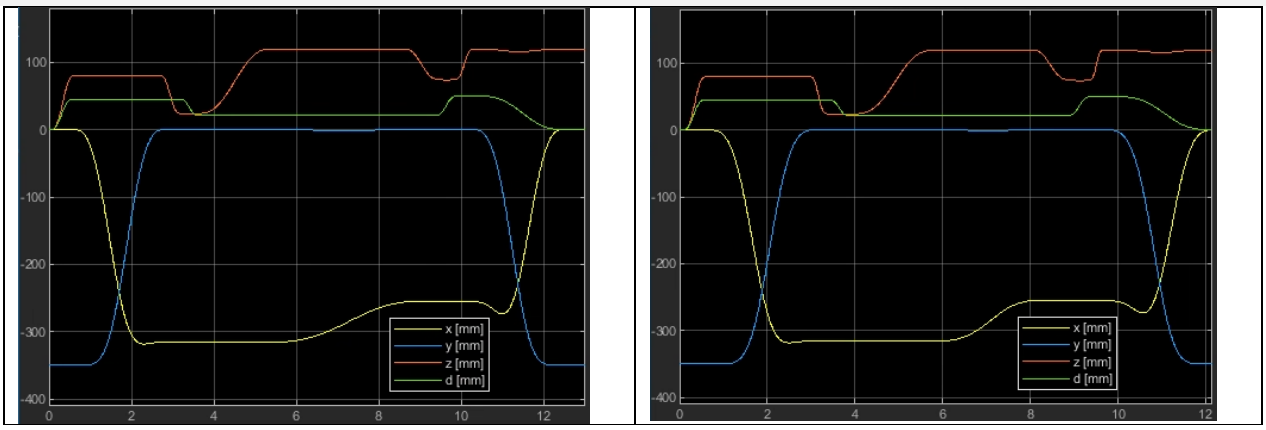
- możliwość wykonania zadania z zadaną maksymalną prędkością ruchu manipulatora – często może to być wymóg BHP,
- możliwość wykonania zadania w określonym oknie czasowym – dużą rolę w tej kwestii odgrywa dobór algorytmu sterowania, w tym funkcji ruchu,
- możliwość wykonania zadania z założonymi napędami charakteryzującymi się pewną dynamiką – odpowiedź na pytanie czy konkretny napęd może zapewnić wyznaczoną prędkość/przyśpieszenie dla analizowanego przegubu.

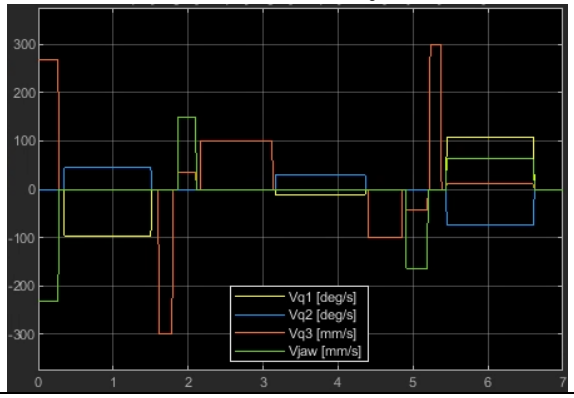
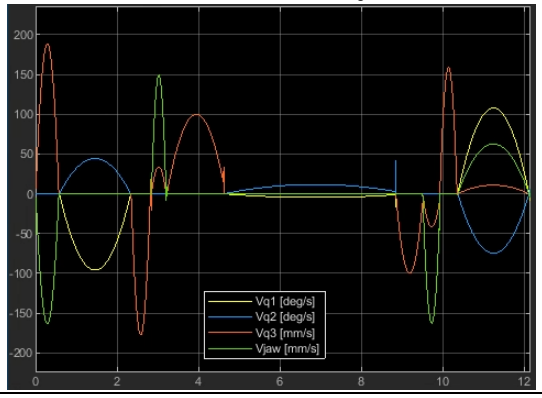
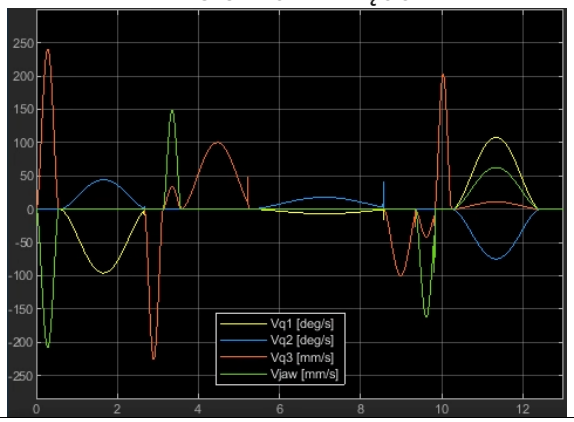
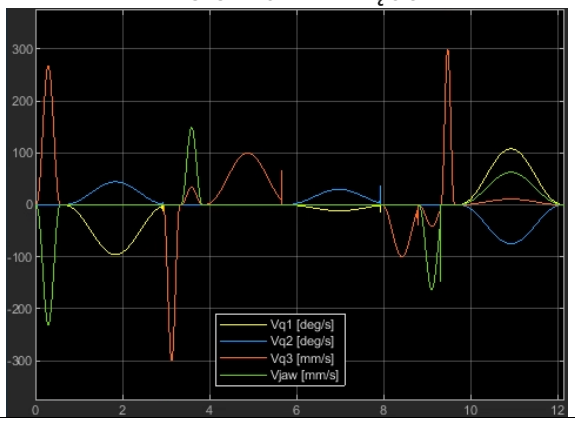
Kolejnym krokiem analizy projektu mogłaby być analiza sił i momentów reakcji w poszczególnych przegubach, pozwalających stwierdzić prawidłowość wyboru napędów pod względem ich charakterystyki dynamicznej. Co więcej, napęd może spełniać postawione wymagania w zależności od wybranego algorytmu sterowania, w tym funkcji ruchu.

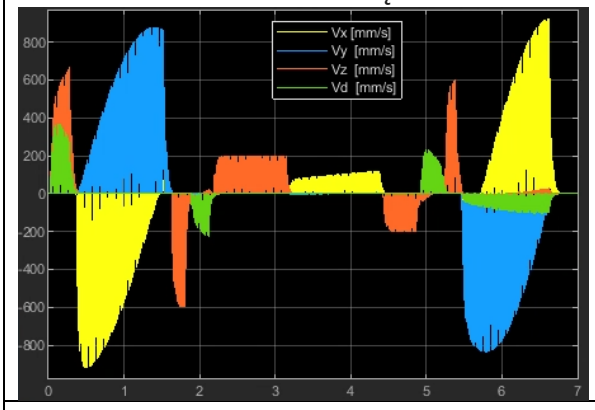
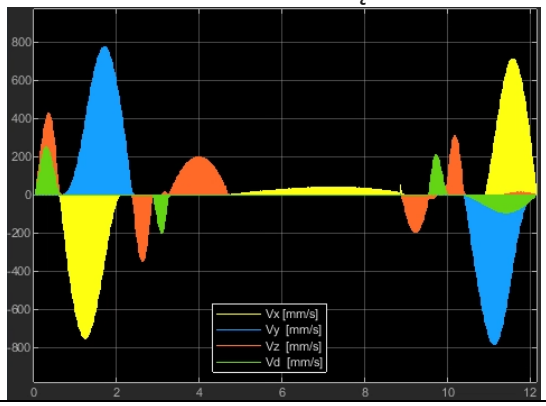
Przykładowe charakterystyki dla zadania ustawienia jednego pionu na drugim:

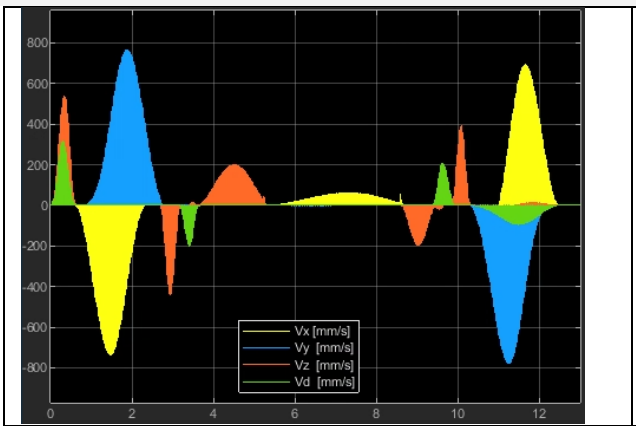
Wartości parametrów przegubowych	
Wielomian II rzędu	Wielomian III rzędu
	
Wielomian V rzędu	Wielomian VII rzędu
	

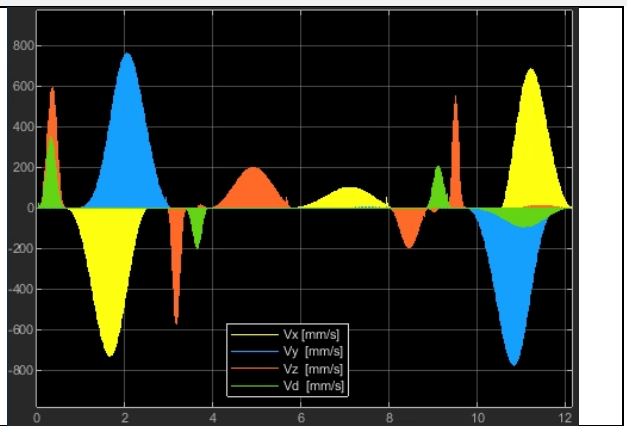
Pozycja TCP	
Wielomian II rzędu	Wielomian III rzędu
	
Wielomian V rzędu	Wielomian VII rzędu



Prędkość dla parametrów przegubowych	
Wielomian II rzędu	Wielomian III rzędu
	
Wielomian V rzędu	Wielomian VII rzędu
	

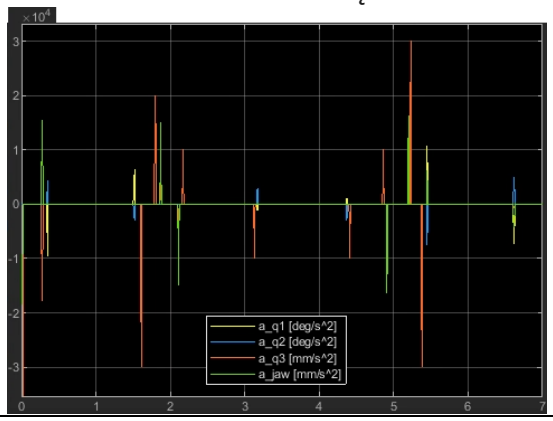
Prędkość TCP	
Wielomian II rzędu	Wielomian III rzędu
	
Wielomian V rzędu	Wielomian VII rzędu



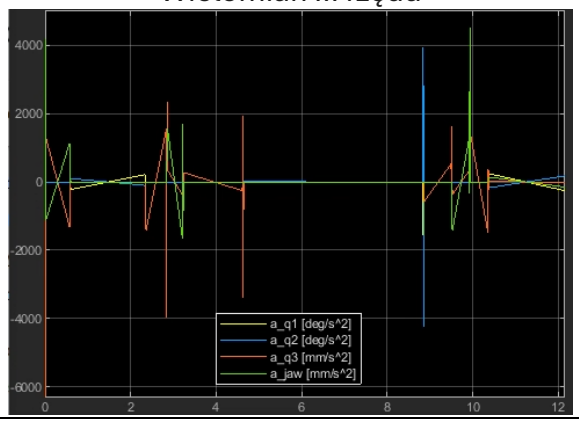


Przyśpieszenie dla parametrów przegubowych

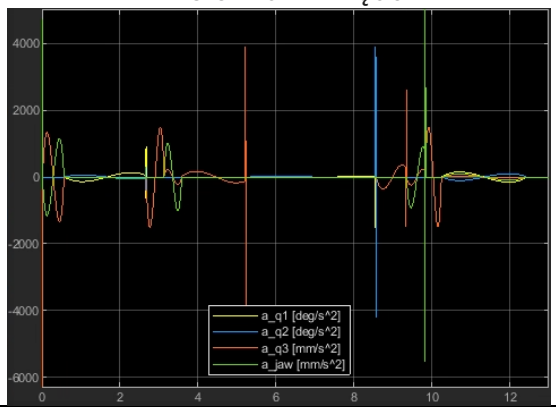
Wielomian II rzędu



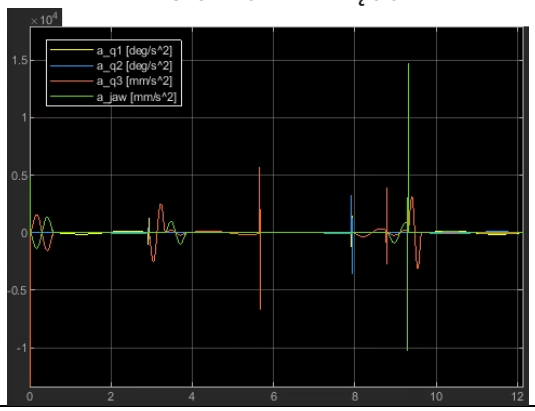
Wielomian III rzędu



Wielomian V rzędu

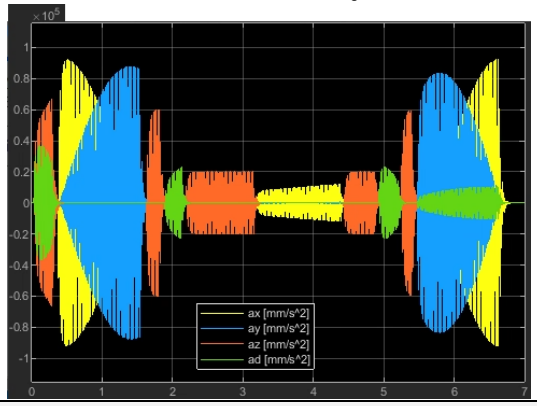


Wielomian VII rzędu

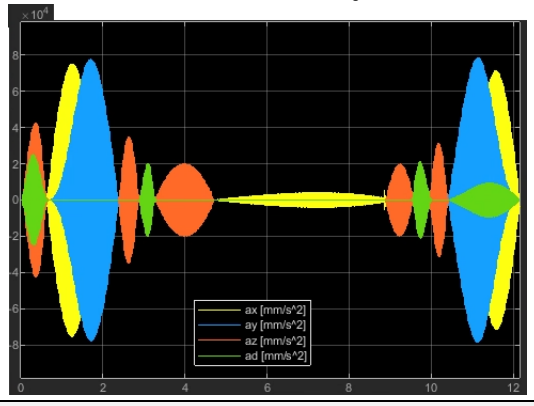


Przyśpieszenie TCP

Wielomian II rzędu



Wielomian III rzędu



Wielomian V rzędu	Wielomian VII rzędu
